

ALTER TABLE: cómo modificar tablas sin perder datos

Crear una tabla bien diseñada no significa que esa tabla va a quedarse igual para siempre. En la práctica, las bases de datos cambian: el negocio crece, aparecen nuevos requisitos, alguien descubre que falta una columna o que una restricción no se aplicó bien. Eliminar y volver a crear la tabla no es una opción cuando ya tiene datos adentro.

Para eso existe **ALTER TABLE**: el comando que te permite modificar una tabla que ya existe sin tocar los datos que contiene.

Qué puedes hacer con ALTER TABLE

Con este comando puedes:

- Agregar columnas nuevas
- Cambiar el tipo de dato de una columna
- Renombrar columnas
- Agregar o eliminar restricciones
- Eliminar columnas
- Renombrar la tabla completa

Cada operación usa una variante distinta del mismo comando. La estructura siempre es la misma: **ALTER TABLE**, el nombre de la tabla, y luego la acción específica.

Agregar una columna nueva

Es la operación más común. Alguien olvidó un campo al crear la tabla y hay que agregarlo sin perder nada de lo que ya está.

```
SQL
ALTER TABLE empleados_demo
ADD COLUMN email TEXT;
```

SQL

```
ALTER TABLE empleados_demo  
  
ADD COLUMN fecha_ingreso DATE NOT NULL DEFAULT CURRENT_DATE;
```

SQL

```
ALTER TABLE empleados_demo  
  
ADD COLUMN activo BOOLEAN NOT NULL DEFAULT TRUE;
```

Cada **ALTER TABLE** agrega una columna. Puedes ejecutarlos uno por uno, en el orden que necesites. Si la columna tiene **NOT NULL**, es obligatorio darle un **DEFAULT**, porque las filas que ya existen en la tabla necesitan un valor para ese campo nuevo.

Cambiar el tipo de dato de una columna

Si una columna se creó con el tipo equivocado, puedes corregirlo sin recrear la tabla.

SQL

```
ALTER TABLE empleados_demo  
  
ALTER COLUMN email TYPE VARCHAR(100);
```

En este caso, el campo **email** se creó como **TEXT** pero debía ser **VARCHAR(100)**. **ALTER COLUMN** seguido de **TYPE** hace el cambio.

Ten en cuenta que cambiar el tipo de dato puede fallar si los datos que ya están en la columna no son compatibles con el nuevo tipo. Por ejemplo, no puedes cambiar una columna de **TEXT** a **INTEGER** si ya tiene letras guardadas.

Renombrar una columna

Cuando el nombre de una columna ya no refleja bien lo que contiene, puedes cambiarlo con **RENAME COLUMN**.

SQL

```
ALTER TABLE empleados_demo  
  
RENAME COLUMN cargo TO puesto;
```

A partir de ese momento, cualquier consulta que use **cargo** va a fallar. Si hay código o reportes que referencian ese nombre, hay que actualizarlos también.

Agregar y eliminar restricciones

Las restricciones también se pueden modificar después de crear la tabla. Para agregarlas se usa **ADD CONSTRAINT**; para eliminarlas, **DROP CONSTRAINT**.

Agregar una restricción:

SQL

```
ALTER TABLE empleados_demo  
  
ADD CONSTRAINT email_unico UNIQUE (email);
```

Esto le pone un nombre a la restricción (**email_unico**) y la aplica al campo **email**. Darle un nombre es importante: es lo que necesitas para poder eliminarla después.

Eliminar una restricción:

SQL

```
ALTER TABLE empleados_demo  
  
DROP CONSTRAINT email_unico;
```

Eliminar una columna

Si una columna ya no sirve, puedes eliminarla con **DROP COLUMN**.

```
SQL
ALTER TABLE empleados_demo
DROP COLUMN activo;
```

Esta operación es **irreversible**. No hay deshacer en SQL. Si eliminas una columna con datos adentro, esos datos desaparecen para siempre. Antes de ejecutar un **DROP**, asegúrate de que realmente no los necesitas o de tener un respaldo.

Renombrar la tabla completa

Si la tabla necesita un nombre nuevo, también se puede cambiar con **ALTER TABLE**.

```
SQL
ALTER TABLE empleados_demo
RENAME TO empleados_v2;
```

A partir de ese momento, la tabla responde al nuevo nombre. Cualquier consulta que use el nombre anterior va a devolver un error.

Resumen de los comandos

Qué quieres hacer	Comando
Agregar una columna	<code>ADD COLUMN nombre tipo</code>
Cambiar el tipo de una columna	<code>ALTER COLUMN nombre TYPE nuevo_tipo</code>
Renombrar una columna	<code>RENAME COLUMN viejo TO nuevo</code>
Agregar una restricción	<code>ADD CONSTRAINT nombre_restriccion TIPO (columna)</code>
Eliminar una restricción	<code>DROP CONSTRAINT nombre_restriccion</code>
Eliminar una columna	<code>DROP COLUMN nombre</code>
Renombrar la tabla	<code>RENAME TO nuevo_nombre</code>

Una regla para usar ALTER TABLE con responsabilidad

`ALTER TABLE` es poderoso porque actúa directamente sobre la estructura de la base de datos. Pero eso también significa que algunos cambios no se pueden deshacer.

Antes de ejecutar cualquier `DROP`, ya sea de columna o de restricción, hazte estas preguntas:

- ¿Hay datos en esa columna que alguien pueda necesitar?
- ¿Hay consultas o reportes que dependan de ese campo o restricción?
- ¿Tienes un respaldo del estado actual de la tabla?

Agregar cosas es seguro. Eliminar cosas requiere estar seguro.

Con la estructura de las tablas dominada, el siguiente paso es empezar a trabajar con los datos que viven dentro de ellas: cómo insertarlos, cómo consultarlos y cómo filtrarlos para obtener exactamente la información que necesitas.